

Lesson 9, Part 2 Activity: Estimate One Slice of Your Project

CS 5388 / CS 4390 Software Project Management | Team activity, complete together

Team: Team 7

Project: Festivo Party Planner

What this activity is for

Your Sprint 2 business case has to answer a question every sponsor asks: how much work would this take, and what would it cost? You are not going to build the system in this course. You are doing what a project manager does before a project is approved: estimating the work so a decision can be made.

Estimating a whole project at once hides mistakes. So you will start small. Pick one or two major parts of your project, called a slice, and estimate only that slice. A small slice lets you count everything, check every number, and see clearly where your assumptions matter.

You will estimate the same slice twice, using two methods that start from different information. Then you will compare the two answers, explain why they differ, and present a range with your assumptions written down. That range becomes the Estimation Appendix on your Sprint 2 portal page.

What to bring

1. Your Release 1 capability list (Step 1 of the Part 1 starter).
2. Your backlog from the Part 1 activity: user stories, MoSCoW categories, and planning poker point sizes.
3. Your similar-products research from Part 1 (small and big products, with timelines and team sizes).
4. The Part 2 slides, especially Slides 13 to 17 (sizing), 19 to 24 (team size), 25 to 27 (story points), and 31 to 33 (reconciling).

How the activity fits together

Step	What you do	What you end up with
1	Pick your slice	One or two major parts of your project, with their stories, screens, and data listed
2	Choose two estimation methods	Two methods that use different starting information
3	Estimate the slice with each method	Two effort estimates, both in team hours
4	Check the estimates for team size	Confidence that coordination costs are included
5	Compare the two estimates	A written explanation of why they differ
6	Present the result	A dated range, in hours, dollars, and weeks, ready for the business case

Key terms

Term	What it means in this activity
Slice	One or two major parts of your project that a user would recognize, such as “buy a permit.” You estimate only this.
Size	How much software the slice contains, measured in function points, use-case points, or story points.
Effort	Total team hours of work. If 5 people each work 10 hours, that is 50 hours of effort.
Productivity rate	Hours of effort per unit of size, such as 8 hours per function point. The rate drives the whole estimate.
Assumption	Any number you chose rather than measured. Label it, so a reader knows it could change.
Velocity	Story points a team completes per sprint. You have no sprint history yet, so yours is an assumption.
Loaded rate	The full hourly cost of one person: salary plus benefits, overhead, and tools.
Range	A low and a high estimate. Honest estimates are ranges, not single numbers.
O, M, P	Optimistic, most likely, and pessimistic estimates, used in the PERT formula $(O + 4M + P) \div 6$.

About the examples

Every italic example uses the same slice from the fictional Campus Parking System: “Buy a semester permit.” Following one slice through every method shows how the pieces connect. Your own numbers come from your project.

Step 1. Pick your slice

A good slice is a part of your project that a user would recognize and that is useful on its own. It usually covers 3 to 8 user stories from your backlog.

How to choose

1. Start with a Must-have capability that is central to your project. If it did not exist, the project would not make sense.
2. If you pick a second part, make it different in kind, such as one part users interact with and one part that produces reports or connects to another system.
3. Do not pick the easiest part. The slice should include some real complexity, such as payments, another system, or stored data.
4. Write down what is out of the slice, so your estimate has a clear boundary.

Question	Our slice
Slice name (one or two major parts)	1. Sourcing data on relevant businesses and vendors 2. Facilitating the RSVP and guest management
Why we chose it	We chose these two parts because vendor research and guest coordination represent two of the central functions of Festivo. Together, they address the project's goal of providing users with one location for finding services and coordinating attendance for an event
Screens or forms where users enter data	Search bar, filters, location, event type, and service/category selection
Reports, results, or lookups users see	A list of relevant businesses and vendors based on the users search, including information such as services, location, pricing, and contact information
Data the slice stores	1. Vendor information including vendor names, addresses, services, pricing and contact information, 2. Guest information including names, invitations and RSVP status. 3. Saved vendor selections associated with an even
Other systems it connects to	Maps/location services and outside business/vendor information
Out of the slice (not included)	Booking vendors, making payments, user reviews, messaging vendors, and other event planning features that are not directly related to finding businesses or invitation/RSVP management.

Stories in the slice

User story	MoSCoW	Points
Search Vendors	Must	8
View Vendor Information	Should	5
Save vendor	Could	3
Send Invitations	Must	8

User story	MoSCoW	Points
View invitation status	Should	5
Notify Guests	Could	2
Total points		31

Example slice, “Buy a semester permit”: a student registers a vehicle (3 points), chooses a lot and permit type (5), pays online (8), receives a digital permit by email (3), and looks up the permit later (2), for 21 points. It stores permit and vehicle records and connects to the student registry and a payment service. Citations and appeals are out of the slice.

Step 2. Choose two methods

Each method starts from different information. Choose two that do not share the same starting point, so that if they agree, the agreement means something (Part 2, Slide 31).

Method	It starts from	A good choice if	Watch out for
Analogous	Similar products and how long they took	You found at least one product close to yours	Copying a big company's timeline without adjusting
Function points	Counts of what users enter, see, and store	Your Step 1 table lists screens, results, and data	A productivity rate with no source
Use-case points	The actors and use cases in your slice	You can list who uses the slice and for what	Default adjustment factors used without saying so
Story points to cost	Your planning poker points and a velocity	Every story in the slice has points	Velocity is a guess until you run sprints

Suggested pairs

1. Function points plus analogous: one starts from your own design, the other from the market. The examples below use this pair.
2. Story points plus analogous: one starts from your backlog, the other from the market.
3. Function points plus story points: one starts from system features, the other from team judgment of story size.

Our two methods: 1. Story Points 2. Use-Case Points

Why these two use different starting information:

Story points start from how our team estimated the amount of work each user story would take. Use case points look more at who is using festivo and what they are using it for. This gives us two different ways to estimate the work needed for our project.

Step 3. Estimate the slice

Complete only the two sections (3A to 3D) that match your methods. Whatever method you use, finish with effort in team hours for your slice, so the two estimates can be compared in Step 5. Where you must choose a number, write it down and label it an assumption.

3B. Use-case points (Part 2, Slides 16 and 17)

What it measures: size based on who uses the slice (actors) and what they do with it (use cases). A use case is a goal a user completes, such as “purchase a permit.” Its steps are the back-and-forth exchanges between the user and the system.

How to do it

1. List your actors. A person using a screen is complex (3). Another system connected through a protocol is average (2). Another system connected through an API is simple (1). Add the weights to get UAW.
2. List the use cases in your slice and count the steps in each. 1 to 3 steps is simple (5), 4 to 7 is average (10), 8 or more is complex (15). Add the weights to get UUCW.
3. Add UAW and UUCW to get unadjusted use-case points (UUCP).
4. The technical factor (TCF) and environment factor (ECF) adjust for difficulty and team experience. If you do not score the full factor lists, use 1.0 for each and write that down as an assumption.
5. Multiply to get UCP, then multiply by a rate. Karner’s starting rate is 20 hours per UCP; label it an assumption.

Actors	Actor Weight (UAW)	Use Case	Weight (UUCW)
Customer	3	Search Vendors	15
Guest	3	View Vendor Information	10
Vendor	2	Save Vendor	5
Admin	3	Send Invitations	10
		View Invitation Status	5
		Notify Guests	5

Item	Calculation	Value
Actor weight (UAW)	Sum of actor weights	11
Use case weight (UUCW)	Sum of use case weights	50
Unadjusted (UUCP)	UAW + UUCW	61
Technical factor (TCF)	1.0 unless you scored it	.61
Environment factor (ECF)	1.0 unless you scored it	1.37
Use-case points (UCP)	$UUCP \times TCF \times ECF$	50.98
Rate (hours per UCP)	20 unless you have a source	20
Effort (hours)	$UCP \times \text{rate}$	1019.6

Example: the student is a complex actor (3); the registry and payment service connect by API (1 each), so $UAW = 5$. Register vehicle is simple (5), purchase permit has 8 or more steps (15), and look up permit is simple (5), so $UUCW = 25$. $UCP = 30$ with factors of 1.0 (assumed). At 20 hours each, effort is 600 hours.

3C. Story points to cost (Part 2, Slides 25 to 27)

What it measures: effort based on your team's own sizing of the stories in the slice. Story points are relative, so you convert them using a velocity (points per sprint). Because you have not run any sprints, velocity is an assumption. Use a low, middle, and high value.

How to do it

1. Use the total points from your Step 1 story table.
2. Choose a low, middle, and high velocity in points per two-week sprint. Label them assumptions.
3. Divide the slice points by each velocity. Keep the decimals; a small slice may take less than one sprint.
4. Multiply by team hours per sprint (team size $\times$ hours per person per sprint) to get effort in hours.

Input	Value	Note
Slice points	31 points	From your Step 1 story table
Team size	5 members	Five or six at most
Hours per person per sprint	16hr/person/sprint	State your assumption
Velocity: low / middle / high	5/8/11	Assumptions until you have sprint data
Sprints needed (high to low velocity)	2.82/3.875/6.2	Slice points $\div$ velocity
Effort in hours (low to high)	225.6/310/496	Sprints $\times$ team size $\times$ hours per person per sprint

Example: 21 points at 30, 20, or 15 points per sprint takes 0.7, 1.05, or 1.4 sprints. With 5 people working 80 hours each per sprint (400 team hours), effort is 280 to 560 hours.

Before you move on: put both estimates in hours

Step 5 compares your two estimates directly, so both must be in team hours for the same slice. If an estimate came out in months, sprints, or dollars, convert it now and show the conversion.

Step 4. Check your estimates for team size (Part 2, Slides 19 to 24)

Larger teams spend more of their time coordinating: meetings, handoffs, and fitting work back together. Estimators tend to underweight this cost, so check each estimate against the list below. Anything marked No needs a fix or a written explanation.

Check	Yes / No	Why it matters
Our team is five or six people at most.	Yes	Beyond six, split into two teams.
We counted our communication paths: $n(n - 1) \div 2 = 10$	Yes	Five people share 10 paths; ten share 45.
We added explicit work for integrating the pieces of the slice.	Yes	Breaking work into pieces hides the cost of putting it back together.
We did not assume that more people means proportionally more output.	Yes	Coordination grows faster than headcount.
We estimated for the team we have.	Yes	Not for a larger team we wish we had.
We adjusted, not copied, timelines from much larger teams.	Yes	Their effort includes their coordination costs.

Step 5. Compare your two estimates (Part 2, Slides 31 and 32)

Two estimates almost never match. The gap is useful: it points to the assumption you are least sure about. Your job is to find that assumption and name it.

	Method 1	Method 2
Method	Story Points	Use-Case Points
Estimate in hours (low to high)	225.6/310/496	1019.6
Middle value in hours	310	N/A
Input that drives it most	Hours per sprint	UUCW
Is that input measured or assumed?	Assumed	Assumed

Gap: larger middle value $\div$ smaller middle value = **1019.6/310 = 3.29** _____

If one estimate is more than about twice the other, recheck your counts and conversions before explaining the gap.

Questions to answer

1. Do your two methods depend on the same assumption? If so, their agreement proves little.

No, the story point estimate depends primarily on our assumed team velocity and productive hours per sprint, while the user-case point estimate depends on the actors, use-case complexity, adjustment factors, and assumed hours per UCP.

2. Which single input moves the answer most? Try changing it and see.

For the story point method, velocity has major effect because changing the assumed points completed per sprint changes the number of sprints required. For the user-case points, the UUCW has major effect because the assigned complexity of each use case directly changes the number of use case points.

3. What information would narrow the gap, and when could you get it?

Actual sprint data would give us measured team velocity instead of an assumed value. Further defining the steps and actors involved in each use case would also import the use-case point estimate. This information would become available as the system requirements and implementation details are refined and development sprints are completed.

Our explanation of the gap (one paragraph)

The two estimation methods produced a significant gap, with the use-case point estimate approximately 3.29 times the middle story point estimate. The methods rely on different assumptions. The story point estimate is strongly affected by our assumed team velocity because we do not yet have historical sprint data, while the use-case point estimate is affected by the number and complexity of actors and use cases. The use-case method also uses an assumed productivity rate of 20 hours per UCP. The gap could be narrowed after completing development sprints and obtaining an actual team velocity, as well as further defining the actors and steps involved in each use case.

Example: function points gave a middle value of 472 hours and the analogous method gave 576 hours, a gap of about 1.2. The two are fairly close, and they start from different information, so the agreement is meaningful. The

largest uncertainty is the productivity rate, which is our assumption, and the fraction of ParkEasy's release that our slice represents.

Step 6. Present the estimate (Part 2, Slide 33)

This is the table your sponsor reads. It gives a range instead of a single number, shows what the range depends on, and is dated because estimates change as you learn more. It becomes the Estimation Appendix on your Sprint 2 portal page.

How to fill it in

1. Range: the lowest and highest hours across both methods.
2. Expected value: set O to your low end and P to your high end. Choose M, your most likely value, somewhere between them and explain why. Then calculate $(O + 4M + P) \div 6$.
3. Cost: multiply hours by a loaded hourly rate. State the rate and where it came from, such as published wage data plus an overhead percentage you choose.
4. Calendar time: divide hours by (team size $\times$ productive hours per person per week). State the productive hours you assume.

Field	Our estimate
Slice estimated	1. Sourcing data on relevant businesses and vendors 2. Facilitating the RSVP and guest management
Range in hours (low to high)	225.6 - 1019.6
Expected hours: $(O + 4M + P) \div 6$, and why we chose M	$O = 225.6, M = 700, P = 1019.6$ $\text{Expected hours} = (225.6 + 4(700) + 1019.6) \div 6 = 674.2 \text{ hrs}$ We chose 700 hours as the most likely value because the two estimation methods produced a larger difference, and several important inputs are still assumption. A value toward the higher end accounts for more uncertainty in team velocity, use case complexity, integration and testing without being overly pessimistic.
Cost range and loaded rate used	Loaded rate: \$73/hour. Cost range = \$16,468.8 - \$74,430.8 Expected cost = $\$73 * 674.2 = \$49,216.6$
Calendar estimate in weeks, and hours per week assumed	Assuming 8 productive hours per person per week, the five-person team provides 40 productive team hours per week. The expected 674.2 hours would require approximately 16.9 weeks, with the full estimate ranging from approximately 5.6 to 25.5 weeks.
Methods used	Story Points, Use Case Points
Key assumptions	Story point velocities of 5, 8, and 11 points per two-week sprint are assumed because the team does not yet have historical sprint data. Each member is assumed to contribute 16 productive hours per sprint. For use-case points, the TCF and ECF values and productivity rate are assumed values. The final estimate also assumes a five-person development team and that the selected slice remains unchanged.
Team size assumed	5 members

Field	Our estimate
What would change this estimate	The estimate would change if actual team velocity differs from our assumptions, use cases or actors are added or removed, technical requirements change, external vendor-data integration becomes more complex, or the scope of the selected slice changes. Actual sprint productivity data would allow us to replace several assumptions with measured values
Date of this estimate	September 27

Source referenced for loaded rate estimation: <https://golance.com/hiring/best-freelance-software-developers-hourly-rate>

Example, “Buy a semester permit” slice: range 354 to 708 hours. With $M = 520$, expected hours = $(354 + 2,080 + 708) \div 6 \approx 524$. At an assumed loaded rate of \$60 per hour, cost is about \$21,000 to \$42,500. With 5 people at an assumed 30 productive hours a week (150 team hours), the expected 524 hours is about 3.5 weeks.

Going further (optional)

Once your slice estimate is solid, you can use it to make a rough estimate of the whole project. If your slice holds about one fifth of your backlog points, the whole project is roughly five times the slice. Treat that number as a first rough guess, and repeat this activity on more slices as you learn more.

Before you submit

1. Your slice is named, with its stories, screens, data, and boundary written down.
2. Both estimates are for the same slice and in hours, with the conversions shown.
3. Every assumed number is labeled as an assumption.
4. Every sourced number has a source you can link.
5. The team-size check is complete, and any No has a fix or explanation.
6. The final estimate is a dated range, not a single number.
7. The individual estimation memo is your own work, written without AI.